

Grid Search and Random Search: Overview

Grid Search and **Random Search** are two popular hyperparameter optimization techniques used to fine-tune machine learning models by searching for the best set of hyperparameters. Here's an explanation of each, along with an example:

1. Grid Search

Grid Search systematically searches through a predefined grid of hyperparameters. It evaluates the model for every possible combination of the provided hyperparameter values.

How Grid Search Works:

- You define a grid of hyperparameters, where each hyperparameter is associated with a list of possible values.
- Grid Search then trains and evaluates the model for every combination of these values.
- It returns the combination that gives the best performance according to a specified metric (e.g., accuracy, mean squared error).

2. Random Search

Random Search, instead of exhaustively searching through the grid, samples a fixed number of hyperparameter combinations randomly from the defined search space. It's less thorough than Grid Search but can be more efficient, especially when the hyperparameter space is large.

How Random Search Works:

- You define a distribution or list of possible values for each hyperparameter.
- Random Search samples random combinations from these distributions a specified number of times.
- The model is trained and evaluated for each sampled combination.
- The combination that gives the best performance is returned.

Lab: Hyperparameter Tuning for Regression Using the Iris Dataset

Objective:

Perform hyperparameter tuning on a selected regression model using Grid Search or Random Search techniques.

Dataset:

Iris dataset, which consists of features such as sepal length, sepal width, petal length, and petal width.

Tools:

- Python
- pandas for data manipulation
- scikit-learn for machine learning algorithms and hyperparameter tuning
- numpy for arrays

Step 1: Import Libraries and Load Data

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import GridSearchCV, RandomizedSearchCV
import numpy as np

# Load Iris dataset

iris = load_iris()
data = pd.DataFrame(data=iris.data, columns=iris.feature_names)

# Assuming we are using petal length as a feature and not as the target,
let's correctly set the target
# Convert the target to float for regression purposes

data['target'] = iris.target.astype(float)

# Features and target variable
X = data.drop('target', axis=1)
y = data['target']

# Check for NaN or infinite values in X and y, and handle them

X = np.nan_to_num(X, nan=0.0, posinf=np.finfo(np.float64).max,
neginf=np.finfo(np.float64).min)
y = np.nan_to_num(y, nan=0.0, posinf=np.finfo(np.float64).max,
neginf=np.finfo(np.float64).min)

# Split the data into training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Step 2: Choose and Train Initial Model

For this lab, we will use RandomForestRegressor as our regression model.

```
# Initialize the RandomForestRegressor
rf_regressor = RandomForestRegressor(random_state=42)

# Train the model on the training data
rf_regressor.fit(X_train, y_train)

# Make predictions
y_pred = rf_regressor.predict(X_test)

# Evaluate initial performance
initial_mse = mean_squared_error(y_test, y_pred)
print(f'Initial Mean Squared Error: {initial_mse:.4f}')
```

Step 3a: Define Hyperparameters for Tuning

We will use GridSearchCV for hyperparameter tuning.

```
# Define hyperparameters for Grid Search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_features': ['sqrt', 'log2', None],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}
```

Step 3b: Define Hyperparameters for Tuning

We will use RandomizedSearchCV for hyperparameter tuning.

```
# Define hyperparameters for Randomized Search
param_dist = {
    'n_estimators': [50, 100, 200, 300],
    'max_features': ['sqrt', 'log2', None],
    'max_depth': [None, 10, 20, 30, 40],
    'min_samples_split': [2, 5, 10, 15],
    'min_samples_leaf': [1, 2, 4, 6]
}
```

Step 4a: Perform Hyperparameter Tuning

Grid Search

```
# Initialize GridSearchCV
grid_search = GridSearchCV(estimator=rf_regressor, param_grid=param_grid,
cv=5, n_jobs=-1, verbose=2)

# Fit GridSearchCV
grid_search.fit(X_train, y_train)

# Best parameters and best score
print(f'Best Parameters (Grid Search): {grid_search.best_params_}')
print(f'Best Score (Grid Search): {grid_search.best_score_:.4f}')
```

```
# Evaluate on test data
best_rf_regressor = grid_search.best_estimator_
y_pred_grid = best_rf_regressor.predict(X_test)
grid_search_mse = mean_squared_error(y_test, y_pred_grid)
```

```
print(f'Grid Search Mean Squared Error: {grid_search_mse:.4f}')
```

Step 4a: Perform Hyperparameter Tuning Random Search

```
# Initialize RandomizedSearchCV
random_search = RandomizedSearchCV(estimator=rf_regressor,
param_distributions=param_dist, n_iter=100, cv=5, n_jobs=-1, verbose=2,
random_state=42)

# Fit RandomizedSearchCV
random_search.fit(X_train, y_train)

# Best parameters and best score
print(f'Best Parameters (Random Search): {random_search.best_params_}')
print(f'Best Score (Random Search): {random_search.best_score_:.4f}')

# Evaluate on test data
best_rf_regressor_random = random_search.best_estimator_
y_pred_random = best_rf_regressor_random.predict(X_test)
random_search_mse = mean_squared_error(y_test, y_pred_random)
print(f'Random Search Mean Squared Error: {random_search_mse:.4f}')
```

Step 5: Conclusion

In this lab, we performed hyperparameter tuning for a regression model using both Grid Search and Random Search techniques. We compared the performance of the models with different hyperparameters and evaluated their effectiveness using Mean Squared Error.

- **Grid Search** provided a detailed exploration of hyperparameter combinations but may be computationally expensive.
- **Random Search** offered a more efficient alternative with potentially less computational cost and comparable results.

In practice, the choice of tuning technique depends on the specific requirements and constraints of the task at hand.